AI-Augmented Startup Pitch Analyzer

Project Documentation

Founders → AI Evaluation → Investors

TEAM MEMBERS:

1 Kushagra Raghav
2 Kritika Gupta

Table of Contents

What is this project?

The AI-Augmented Startup Pitch Analyzer is a web application where founders register, fill in a detailed startup pitch form, upload their PDF pitch deck and company logo, and receive a full AI-generated VC-style evaluation report.

Investors log in to a real-time analytics dashboard, browse the startup pipeline, filter by category, view full AI analysis reports, and take actions like Shortlist / Save / Pass.

The AI engine — powered by OpenAI GPT-4o-mini — reads the uploaded pitch deck, analyzes the business, and returns structured investment intelligence: scores, strengths, weaknesses, competitor analysis, market sizing, and downloadable PDF reports.

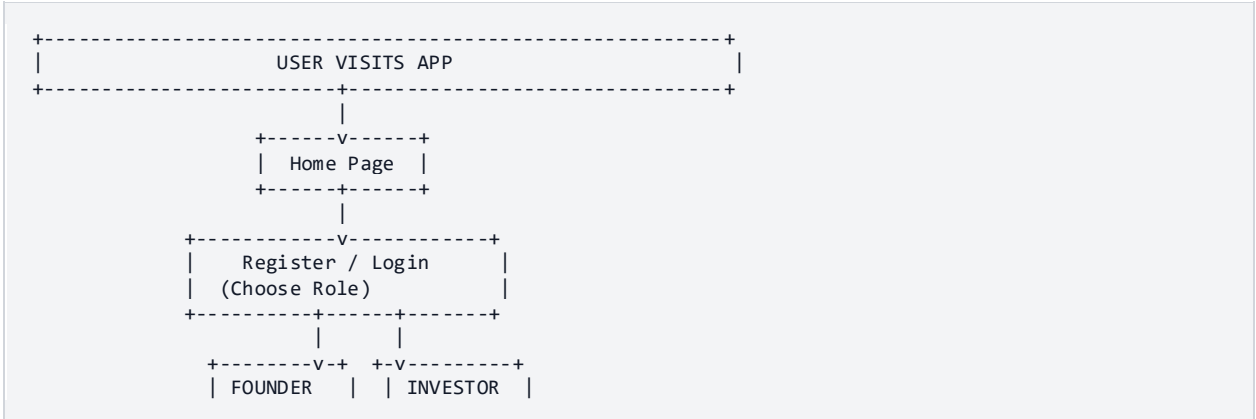
Why was it built? — Problem Statement

Startups often struggle to get early, unbiased feedback on their pitches before approaching investors. Meanwhile, investors receive hundreds of decks with no way to quickly assess quality at scale. This platform solves both sides:

Problem	Solution
Founders spend weeks waiting for investor feedback	AI gives instant VC-level analysis in seconds
Pitch decks are hard to evaluate consistently	Standardized 6-dimension scoring system
Investors can't quickly screen high-potential startups	Real-time analytics dashboard with score filters
No central place to manage startup deal flow	Startup pipeline with Shortlist / Save / Pass actions
Creating evaluation reports is slow and expensive	Auto-generated downloadable PDF reports

How does it work?

Full System Flowchart

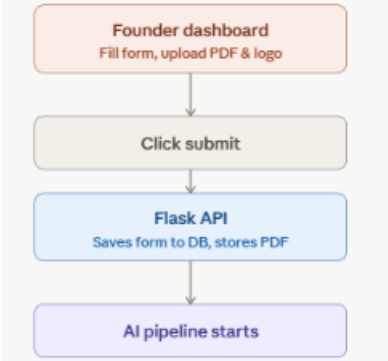


```
      +----+------+  +-----+-----+
           |              |
+----------v------+   +----v-----------------+
| Founder Dashboard|   | Investor Dashboard    |
| - Fill Form      |   | - Browse Startups     |
| - Upload PDF     |   | - Filter / Sort       |
| - Upload Logo    |   | - View AI Reports     |
+----------+-------+   | - Shortlist / Save    |
           |           +-----------------------+
+----------v-----------+
|   Click "Submit"     |
+----------+-----------+
           |
+----------v-----------+
|  Backend: Flask API  |
|  - Save form to DB   |
|  - Store PDF file    |
+----------+-----------+
           |
+----------v-----------+
|  AI Pipeline Starts  |
|  (Auto on submit)    |
+----------+-----------+
           |
+----------v-----------+
|  PyMuPDF extracts    |
|  text from PDF       |
+----------+-----------+
           |
+----------v-----------+
|  OpenAI GPT-4o-mini  |
|  Call 1: VC Analysis |
|  Call 2: Market Intel|
+----------+-----------+
           |
+----------v-----------+
|  Results saved to DB |
|  (PostgreSQL/SQLite) |
+----------+-----------+
           |
+----------v-----------+
|  Dashboard shows:    |
|  Scores + Report     |
|  + Download PDF      |
+----------------------+
```

1. Onboarding and role split

The investor dashboard is mostly a browsing/filtering surface — browse startups, filter and sort, view AI reports, shortlist favorites — with no submission step, so there's nothing further to branch out there. The founder path is where the real pipeline kicks off

2. Founder submission triggering the AI pipeline

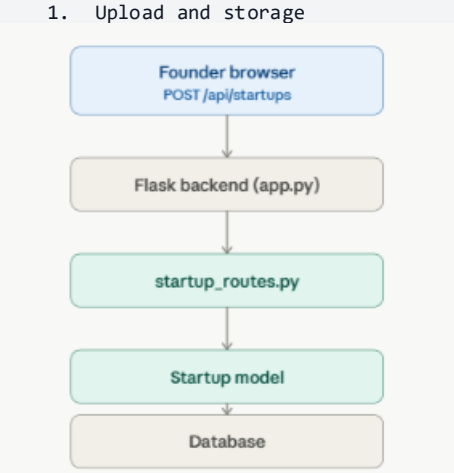


From "AI pipeline starts," the flow continues exactly as the analysis generation and results delivery diagrams above show: PyMuPDF extraction → the two OpenAI calls → saved to DB → rendered back on the dashboard with scores, report, and a downloadable PDF.

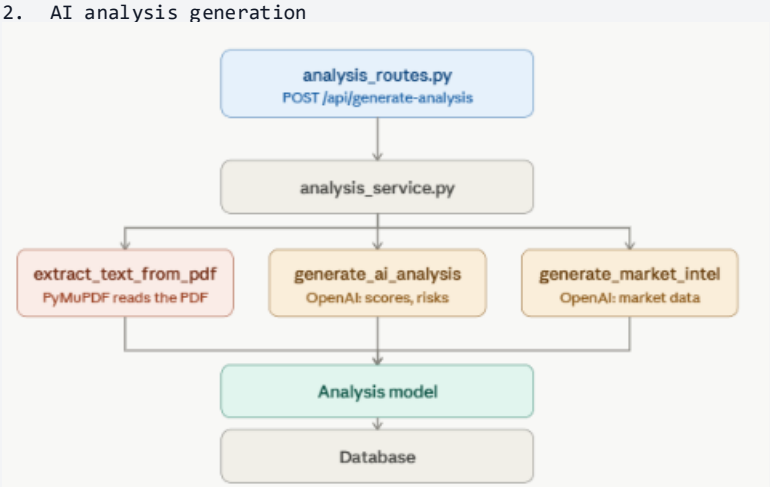
Data Flow Diagram

```
[Founder Browser]
       |
       |  POST /api/startups  (FormData: form fields + PDF + Logo)
       v
[Flask Backend - app.py]
       |
       +---> [startup_routes.py] ---> [Startup Model] ---> [Database]
       |
       |  POST /api/generate-analysis  { startup_id }
       v
[analysis_routes.py]
       |
       v
[analysis_service.py]
       +---> extract_text_from_pdf()     <-- PyMuPDF reads uploaded PDF
       |
       +---> generate_ai_analysis()      <-- OpenAI Call #1
       |          Returns: scores, strengths, weaknesses,
       |                   risks, opportunities,
       |                   investment_recommendation,
       |                   due_diligence_questions
       |
       +---> generate_market_intelligence()  <-- OpenAI Call #2
       |          Returns: competitors, SWOT, TAM/SAM/SOM,
       |                   industry trends, strategic recommendations
       |
       +---> [Analysis Model] ---> [Database]
                  |
                  v
       [GET /api/startups] ---> [React Frontend]
                                      |
                              [Founder Dashboard]
                              [Investor Dashboard]
```

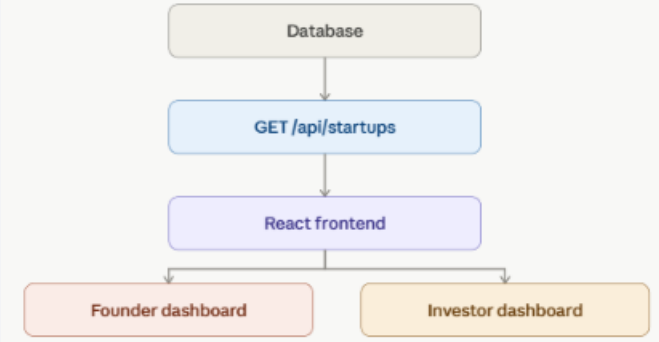
1. Upload and storage



2. AI analysis generation



3. Serving results to the frontend



1. User Login & Role System

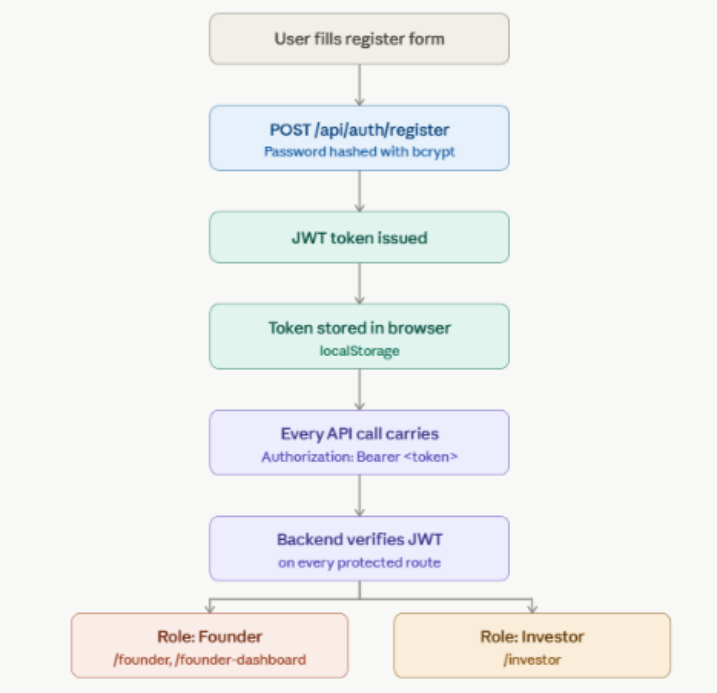
Registration

When signing up, users choose a role:

Role	What they can do
Founder	Submit startups, view own analysis dashboard, download PDF reports
Investor	View all startups, filter by category, view full AI analysis, shortlist/save/reject

Authentication Flow

```
User fills Register Form
         |
         v
POST /api/auth/register  ->  Password hashed with bcrypt
         |
         v
     JWT Token issued
         |
         v
Token stored in browser (localStorage)
         |
         v
All API calls carry:  Authorization: Bearer <token>
         |
         v
Backend verifies JWT on every protected route
         |
         +-- Role = Founder  ->  Access: /founder, /founder-dashboard
         +-- Role = Investor ->  Access: /investor
```



Passwords are hashed using bcrypt before storage.

Authentication uses JWT (JSON Web Tokens) via Flask-JWT-Extended.

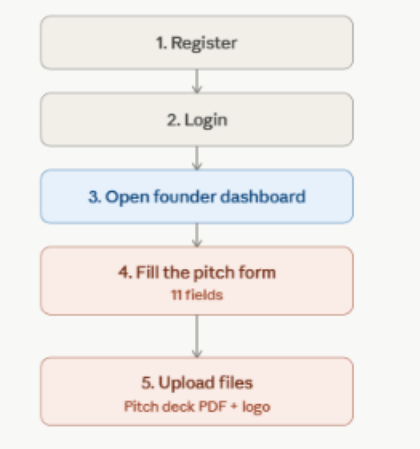
Frontend uses React Context API (AuthContext) to manage login state globally.

ProtectedRoute component guards all private pages — wrong role means access denied.

Founder Journey — Step by Step

```
1. Register  ->  2. Login  ->  3. Open Founder Dashboard
      |
      v
4. Fill the Pitch Form (11 fields)
      |
      v
5. Upload Pitch Deck (PDF) + Company Logo (Image)
      |
      v
6. Click "Save Startup"
      |
      v
7. AI Analysis runs automatically (full-screen loading overlay shown)
      |
      v
8. Dashboard refreshes with full AI evaluation
      |
      v
9. View scores, strengths, weaknesses, risks
      |
      v
10. Click "Download Report" to get a PDF evaluation report
```

Steps 1–5: Getting to a saved pitch



Steps 6–10: From save to a downloadable report

KPI Snapshot (Top of Founder Dashboard)

The dashboard shows a live summary of all submitted startups:

- Total Startups submitted
- Evaluated count (those with AI analysis)
- Average Overall Score across all analyzed startups
- Radar Chart — average score per dimension (Market, Team, Product, Business, Scalability, Financial)

Search and Filter (Founder Dashboard)

- Search bar — find startups by name, market, solution
- Category filter — AI / Fintech / HealthTech / Climate / Other
- Sort — Newest / Oldest / Highest Score / Most Fundable
- Pagination — 12 startups per page

What the Form Collects

When a Founder clicks Submit, the form sends these fields:

Field	Type	Description
startup_name	Text	Name of the startup
industry	Text	e.g., SaaS, HealthTech, FinTech
category	Dropdown	AI / Fintech / HealthTech / Climate / Other
funding_stage	Dropdown	Pre-seed / Seed / Series A / Series B+
funding_ask	Text	e.g., "$500K" or "$2M"
problem	Textarea	What problem the startup solves
solution	Textarea	How the startup solves it
target_market	Textarea	Who are the customers
business_model	Textarea	How the startup makes money
traction	Textarea	Revenue, users, partnerships, milestones
description	Textarea	General overview / elevator pitch
pitch_deck	File (PDF)	The actual pitch deck — this is what the AI reads
logo	File (Image)	Company logo for display on cards

Supported Startup Categories

AI · Fintech · HealthTech · Climate · Healthcare · EdTech · Food · Sports · Cyber Security · Agriculture · Travel · Gaming · SaaS · Logistics · Manufacturing · Fashion · Energy · Other

Supported Funding Stages

Pre-seed · Seed · Series A · Series B+

What Happens After Submit — The AI Pipeline

When the form is submitted with a PDF pitch deck attached, the following automated pipeline runs:

Step 1 — Data Saved to Database

```
FormData  ->  Flask receives multipart/form-data
           ->  Startup record created in DB
           ->  PDF saved to /backend/uploads/
           ->  Logo saved to /backend/uploads/
```

Step 2 — PDF Text Extraction

```
# Using PyMuPDF (fitz library)
extract_text_from_pdf(file_path)
# Opens every page of the PDF
# Extracts all readable text from each page
# Combines all pages into one string
# This text is sent to the AI
```

Step 3 — AI Call #1: VC Analysis (OpenAI GPT-4o-mini)

The extracted pitch deck text is sent to GPT-4o-mini with this role:

```
System: "You are an expert venture capital analyst."
Input:  Full pitch deck text (extracted from PDF)
Output: Structured JSON with scores and qualitative analysis
```

The AI evaluates on 8 dimensions (all scored 1–10):

Dimension	What it measures
Market Score	Market size, growth potential, demand strength
Team Score	Founder experience, team capability
Product Score	Innovation level, product-market fit
Business Score	Revenue model, unit economics
Scalability Score	Ability to grow rapidly and efficiently
Financial Score	Financial health, burn rate, projections
Overall Score	Composite VC rating (headline number)
Funding Readiness Score	How investor-ready the startup is

Step 4 — AI Call #2: Market Intelligence (OpenAI GPT-4o-mini)

A second AI call analyzes the market landscape:

```
System: "You are a market intelligence researcher."
Input:  Same pitch deck text
Output: Competitive landscape and market sizing data
```

Step 5 — Results Saved and Dashboard Updated

All AI results are stored in the analyses table in the database. The frontend automatically fetches the updated data and shows the full evaluation on the dashboard.

What the AI Gives Back

After both AI calls complete, the system provides 15+ distinct outputs.

Scores (8 numeric values — each rated 1 to 10)

```
Overall Score           <- Headline rating for the startup
Market Score            <- Market opportunity and size
Team Score               <- Founding team strength
Product Score            <- Product innovation and PMF
Business Score           <- Business model strength
Scalability Score        <- Growth ceiling and potential
Financial Score          <- Financial health and projections
Funding Readiness Score  <- Investor readiness level
```

Qualitative Analysis

Output	Description
Executive Summary	AI-written paragraph overview of the startup
Strengths	Bullet list of key competitive advantages
Weaknesses	Bullet list of gaps or concerns
Opportunities	Market gaps the startup can exploit
Risks	Execution, market, or competitive risks
Investment Recommendation	Final VC verdict — a paragraph advice
Due Diligence Questions	Questions an investor should ask before writing a check

Market Intelligence

Output	Description
Direct Competitors	Companies solving the exact same problem
Indirect Competitors	Adjacent or partial alternatives
SWOT Analysis	Full Strengths / Weaknesses / Opportunities / Threats grid
TAM	Total Addressable Market — the full market size
SAM	Serviceable Addressable Market — reachable segment
SOM	Serviceable Obtainable Market — realistic near-term target
Industry Trends	What is trending in this sector right now

Output	Description
Emerging Technologies	Relevant technologies to watch
Consumer Trends	How customer behavior is shifting
Growth Opportunities	Expansion and upsell paths
Strategic Recommendations	Concrete advice on how to scale the startup

Downloadable PDF Report

Clicking “Download Report” on any analyzed startup generates a full PDF containing:

```
Page 1:  Startup name, industry, category, funding stage
         Executive Summary
         Score Summary (all 8 scores)
Page 2+: Strengths
         Weaknesses
         Opportunities
         Risks
         Due Diligence Questions
         Market Intelligence (Competitors, TAM/SAM/SOM,
         Industry Trends, Consumer Trends,
         Growth Opportunities, Strategic Recommendations)
```

Investor Dashboard Features

The Investor Dashboard is a real-time analytics and deal flow platform.

KPI Cards (4 headline metrics at the top)

```
+------------------+ +------------------+ +------------------+ +------------------+
|  Total Startups  | |    Analyzed      | |   Avg Score      | | High Potential   |
|   In Pipeline    | |  With AI eval    | |   Overall /10    | |   (Score 7+)     |
+------------------+ +------------------+ +------------------+ +------------------+
```

4 Interactive Charts (built with Chart.js)

Chart	Type	What it shows
Startups by Category	Horizontal Bar Chart	Count of startups per industry category
Startups by Funding Stage	Vertical Bar Chart	Pre-seed vs Seed vs Series A vs Series B+
Average Evaluation Scores	Line Chart	Average AI score across all 6 dimensions
Category Mix	Doughnut Chart	Portfolio diversification by sector

Startup Pipeline (Scrollable List)

Each startup card in the pipeline shows:

- Startup name, industry, category tag
- Funding ask amount (badge)
- Overall AI score badge (e.g., "Score: 8/10")
- "High Potential" green badge (automatically shown if score is 7 or above)
- Current investor action status (if any action was taken)

Per-Startup Actions

Button	What it does
View Details	Opens full startup profile in the sidebar panel
Pitch Deck	Downloads and opens the original uploaded PDF
Shortlist	Marks startup as shortlisted for deeper review
Save	Saves for later consideration
Pass	Marks as rejected / passed

Startup Profile Sidebar

When you click "View Details", a sidebar opens showing:

- Name, category, industry
- Funding ask
- Problem statement
- Solution description
- Traction details
- Overall AI score (highlighted panel)

Portfolio Stats Sidebar

Live counter showing:

- How many startups are Shortlisted
- How many startups are Saved
- How many startups are Analyzed

Filter & Search (Investor Dashboard)

- Category pills — clickable filter buttons for all 15+ categories (All, Healthcare, FinTech, EdTech, AI, SaaS, etc.)
- Search bar — real-time search by startup name or keywords
- Sort dropdown — Newest / Oldest / Highest Score / Most Fundable

Tech Stack

Frontend — React + Vite

Technology	Version	Purpose
React	18.3.1	Component-based UI framework
Vite	5.4.10	Ultra-fast dev server and build tool
React Router v6	6.21.1	Client-side routing and protected routes
Tailwind CSS	3.4.15	Utility-first CSS framework
Chart.js	4.4.0	Data visualization engine
react-chartjs-2	5.2.0	React wrapper for Chart.js
Axios	1.7.4	HTTP client for all API calls
React Context API	built-in	Global authentication state

Charts implemented:

- Radar — Founder dashboard: average dimension scores
- Bar (horizontal) — Investor: startups by category
- Bar (vertical) — Investor: startups by funding stage
- Line — Investor: average evaluation scores across dimensions
- Doughnut — Investor: portfolio category mix

Backend — Flask + Python

Technology	Version	Purpose
Flask	3.0.3	Python web framework
Flask-SQLAlchemy	3.1.1	ORM for database models and queries
Flask-JWT-Extended	4.6.0	JWT-based authentication
Flask-CORS	5.0.0	Cross-origin resource sharing
python-dotenv	1.0.1	.env environment variable loading
bcrypt	4.1.2	Secure password hashing
PyMuPDF (fitz)	1.28.0	PDF text extraction engine
fpdf	1.7.2	PDF report generation
openai	1.0.0	GPT-4o-mini API client

Technology	Version	Purpose
gunicorn	23.0.0	Production WSGI server
psycopg2-binary	2.9.10	PostgreSQL database adapter

Database:

- Development: SQLite (auto-created, zero configuration needed)
- Production: PostgreSQL (configured via DATABASE_URL environment variable)

AI Model: gpt-4o-mini — fast, cost-effective, returns structured JSON

Project Folder Structure

```
internship/
|
+-- backend/                          <- Flask Python API
|   +-- app.py                        <- Main app entry: Flask config, blueprint registration, DB
init
|   +-- extensions.py                 <- Shared db, jwt, cors instances
|   +-- requirements.txt              <- All Python package dependencies
|   +-- runtime.txt                   <- Python version pinned for deployment
|   +-- .env.example                  <- Template for setting up .env
|   |
|   +-- routes/                       <- API route definitions (Flask Blueprints)
|   |   +-- auth_routes.py            <- POST /api/auth/register, POST /api/auth/login
|   |   +-- startup_routes.py         <- GET/POST/PUT/DELETE /api/startups (handles file uploads)
|   |   +-- analysis_routes.py        <- POST /api/generate-analysis, GET /api/analysis/:id/report
|   |   +-- health_routes.py          <- GET /api/health
|   |
|   +-- models/                       <- SQLAlchemy DB models (tables)
|   |   +-- startup.py                <- Startup table: all pitch fields + analysis status
|   |   +-- analysis.py               <- Analysis table: all AI output fields (scores, lists,
strings)
|   |   +-- user.py                   <- User table: id, email, password hash, role
|   |
|   +-- controllers/                  <- Business logic separated from routes
|   +-- services/                     <- Core processing layer
|   |   +-- analysis_service.py       <- PDF extraction + 2x OpenAI calls + PDF report generation
|   |   +-- health_service.py         <- Simple health check
|   |
|   +-- middleware/                   <- Custom Flask middleware (auth guards)
|   +-- utils/                        <- Shared helper functions
|   +-- api/                          <- Additional API helpers
|   +-- uploads/                      <- Storage for uploaded pitch decks and logos
|
+-- src/                              <- React frontend source code
|   +-- App.jsx                       <- Root component: sets up routing + wraps with AuthProvider
|   +-- main.jsx                      <- App entry point (ReactDOM.createRoot)
|   |
|   +-- pages/                        <- Full-page React components
|   |   +-- HomePage.jsx              <- Landing / welcome page
|   |   +-- FounderPage.jsx           <- Founder role selection / landing
|   |   +-- FounderDashboardPage.jsx  <- Main dashboard: pitch form + KPIs + analysis cards
|   |   +-- InvestorPage.jsx          <- Full investor dashboard: charts + pipeline + sidebar
|   |   +-- ProfilePage.jsx           <- User profile settings page
|   |   +-- auth/
|   |       +-- LoginPage.jsx         <- Login form
|   |       +-- RegisterPage.jsx      <- Register form with role selection (Founder / Investor)
|   |
|   +-- components/                   <- Reusable UI building blocks
|   |   +-- Navbar.jsx                <- Top navigation bar with auth-aware links
|   |   +-- ProtectedRoute.jsx        <- HOC that checks JWT + role before rendering page
|   |
|   +-- context/
|   |   +-- AuthContext.jsx           <- React Context: login(), logout(), user state, token
|   |
|   +-- api/
|   |   +-- axios.js                  <- Pre-configured Axios instance (base URL + JWT header)
|   |
|   +-- utils/                        <- Frontend utility/helper functions
|   +-- styles/                       <- Global CSS files
|
+-- index.html                        <- Vite HTML entry point
+-- vite.config.js                    <- Vite build configuration
+-- tailwind.config.js                <- Tailwind CSS customization
+-- postcss.config.js                 <- PostCSS setup for Tailwind
```

```
+-- package.json                    <- Frontend npm dependencies and scripts
```

Local Setup Guide

Prerequisites

- Node.js >= 18
- Python >= 3.9
- OpenAI API Key — get one at platform.openai.com

Step 1 — Clone the Repository

```
git clone https://github.com/YOUR_USERNAME/YOUR_REPO_NAME.git
cd YOUR_REPO_NAME/internship
```

Step 2 — Backend Setup

```
cd backend
# Create virtual environment
python -m venv venv
# Activate it
venv\Scripts\activate          # Windows
# source venv/bin/activate     # macOS / Linux
# Install all Python dependencies
pip install -r requirements.txt
# Create your .env file from the template
copy .env.example .env         # Windows
# cp .env.example .env         # macOS / Linux
# Open .env and fill in your values (see Environment Variables section below)
# Start the Flask development server
flask run
```

Backend runs at: http://localhost:5000

Step 3 — Frontend Setup

```
# Go back to the internship root
cd ..
# Install all Node.js dependencies
npm install
# Start the Vite development server
npm run dev
```

Frontend runs at: http://localhost:5173

API Reference

Authentication

Method	Endpoint	Request Body	Description
POST	/api/auth/register	{ name, email, password, role }	Create a new account
POST	/api/auth/login	{ email, password }	Login and receive a JWT token

Startups

Method	Endpoint	Auth	Description
GET	/api/startups	Yes	List startups — supports ?category, ?search, ?sort_by, ?page, ?page_size
POST	/api/startups	Yes	Create startup — multipart/form-data with PDF and logo files
GET	/api/startups/:id	Yes	Get one startup (includes analysis data)
PUT	/api/startups/:id	Yes	Update startup details
DELETE	/api/startups/:id	Yes	Delete a startup
GET	/api/startups/:id/pitch_deck	Yes	Download the uploaded pitch deck PDF
POST	/api/startups/:id/action	Yes	Investor action — body: { action: "shortlist" / "save" / "reject" }

Analysis

Method	Endpoint	Request Body	Description
POST	/api/generate-analysis	{ startup_id }	Trigger full AI analysis pipeline (PDF extract + 2 AI calls)
GET	/api/analysis/:id/report	—	Download auto-generated PDF evaluation report

Health

Method	Endpoint	Description
GET	/api/health	Returns { "status": "ok" } — used for uptime monitoring

Environment Variables

Create a backend/.env file with the following content:

```
# Flask app entry point
FLASK_APP=app.py
FLASK_ENV=development

# Secret keys - use long random strings in production
SECRET_KEY=your-secret-key-here
JWT_SECRET_KEY=your-jwt-secret-key-here

# Database connection
# For local development (SQLite - no setup needed):
DATABASE_URL=sqlite:///app.db
# For production (PostgreSQL):
# DATABASE_URL=postgresql://user:password@host:5432/dbname

# OpenAI API key for AI analysis
OPENAI_API_KEY=sk-your-openai-api-key-here
```

Note: If OPENAI_API_KEY is missing or set to change-me, the app automatically returns realistic mock/fallback data. You can fully test the UI and flow without an API key.

Contributing

- Fork this repository
- Create your feature branch: git checkout -b feature/my-new-feature
- Commit your changes: git commit -m "Add: description of feature"
- Push to the branch: git push origin feature/my-new-feature
- Open a Pull Request

License

Acknowledgements

Tool	Purpose
OpenAI GPT-4o-mini	AI analysis engine (VC analysis + market intelligence)
Flask	Python backend web framework
React	Frontend UI framework
Vite	Lightning-fast build tool
Chart.js	Interactive data visualizations
PyMuPDF	PDF text extraction
Vercel	Frontend deployment and hosting

Built with love to bridge the gap between startup founders and smart investors.